UNITS

Boba systemd user units

Revision: 2 **Last modified:** 2026-09-01T15:43:15Z **Scope:** `scripts/systemd/user/**` — the `systemctl --user` start path and how it relates to `./start.sh`. **Authority:** feature `002-user-owned-downloads` (FR-008, FR-009, FR-010, FR-016); CLAUDE.md Hard Stop #3; CONST-033.

Why this document exists

The project has two start paths. The operator's own words that opened this feature name the second one explicitly:

> "Make sure that System runs (starts) and creates and modifies files as current user under which we do start it using `systemctl --user` space so downloads we have and directories that have been created can be manipulated by current account user!"

Two start paths that disagree is a defect on its own: one of them is wrong and nobody knows which. This document records the deliberate decision that keeps them from disagreeing, and the guarantees each path carries.

The units

Unit	Type	Role
`boba.target`	target	Umbrella. `Wants=` the stack and the bridge. Enabled into `default.target`.
`boba-stack.service`	oneshot + `RemainAfterExit`	

Unit	Type	Role
		Delegates to `./start.sh --no-build` / `./stop.sh`.
`boba-webui-bridge.service`	simple	Supervises the `webui-bridge` Go binary on port 7188.
`boba-resource-pressure-check.service`	oneshot	Runs the resource-pressure challenge.
`boba-resource-pressure-check.timer`	timer	Fires the check. Into `timers.target`, **not** `boba.target`.

The resource-pressure pair is deliberately outside `boba.target`: host resource-pressure monitoring must keep running when the stack is down. It is still part of the `scripts/boba-svc.sh` install inventory, so `install`/`uninstall` are total.

Lifecycle ownership — the deliberate decision (FR-009)

A strict two-layer split:

- **Outer layer — systemd** owns *session* lifecycle only: autostart at login/boot, stop on logout, journal capture, restart-on-failure. It issues **no container command of any kind**.
- **Inner layer — `start.sh`** owns *container* orchestration exclusively (Hard Stop #3), and is the only place the ownership gate lives.

Three properties follow, and they are what make the two paths safe together:

1. **No unit may contain a raw `podman` / `docker` / compose command.** Every unit delegates to `./start.sh` or `./stop.sh`. Verified: the only occurrences of those words under `scripts/systemd/user/` are in comments, never in a directive.
2. **`start.sh` is idempotent**, so `systemctl --user start boba.target` against an already-running stack *converges* instead of conflicting.
3. **The containers are the single source of truth for "what is running."**

The ownership gate is inherited, never duplicated (FR-010, FR-004d)

`start.sh` runs `run_ownership_gate()` before any container writes into a declared location: the precondition (`scripts/ownership_precondition.sh`, fail-closed on both exit 1 *and* exit 2) followed by the repair (`scripts/ownership_repair.sh`), both under `nice -n 19 ionice -c 3`.

That gate is **not** declared as an `ExecStartPre=` in any unit. It exists in exactly one place and the systemd path inherits it by delegating to `start.sh`. Two copies would be two things that can drift, and a systemd path whose gate had drifted from the `start.sh` path is precisely the contradiction FR-009 forbids. Anything `start.sh` enforces, `systemctl --user start boba.target` enforces — by construction, with no further wiring.

The one divergence that cannot be engineered away, and how it is handled

`boba-stack.service` is `Type=oneshot` + `RemainAfterExit=yes`. systemd's notion of "active" therefore means *"this unit ran `start.sh` once and it exited 0"* — not *"the containers are up."* Start the stack directly with `./start.sh` and systemd keeps reporting `inactive` while the containers are healthy. systemd cannot observe a stack it did not itself start.

That divergence is not hidden. `start.sh` ends by comparing systemd's view with reality and, when they differ, printing the real state, the systemd state, and the single command that reconciles them (`bash scripts/boba-svc.sh up`, which re-runs this same `start.sh`). A divergence the operator is told about is not a contradiction; a silent one is.

Restart bounding

`boba-stack.service` sets `RestartSec=30`, while systemd's default rate-limit window is `10s` / `5` tries. Restarts 30 s apart never accumulate inside a 10 s window, so the default limiter could never trip: a persistently failing `start.sh` would re-run a full stack start every 30 s forever, silently. That matters more now that `start.sh` is fail-closed on the ownership precondition, because a genuine refusal is persistent by

design. The unit sets `StartLimitIntervalSec=600` / `StartLimitBurst=3`, so a real transient still self-heals while a persistent refusal lands in `failed`, where `systemctl --user status` shows it.

CONST-033

No unit here contains any power-state directive, and none may be added. No suspend / hibernate / poweroff / reboot / halt / kexec, and no setting that cascades into an idle action. Verified: the only occurrences of those words under `scripts/systemd/user/` are in prose comments, never in a directive.

Operating the units

```
bash scripts/boba-svc.sh install    # render + copy every unit
        into ~/.config/systemd/user/
bash scripts/boba-svc.sh enable     # autostart at login/boot
        (reports linger state)
bash scripts/boba-svc.sh up         # systemctl --user start
        boba.target
bash scripts/boba-svc.sh status     # systemd view
bash scripts/boba-svc.sh health     # HTTP probes of every
        published endpoint
bash scripts/boba-svc.sh down       # systemctl --user stop
        boba.target
```

No `sudo` at any point. Boot-time autostart without a login additionally needs linger (`loginctl enable-linger <user>`), which `boba-svc` reports but never runs itself.

Unit templating — `@@BOBA_REPO_ROOT@@` (2026-09-01)

The units under `scripts/systemd/user/` are **templates**, not final files. Every reference to the repository root is written as the token `@@BOBA_REPO_ROOT@@`; `boba-svc install` substitutes this checkout's real root and installs the result as a **copy**.

The defect this fixes. Every unit previously hardcoded the absolute prefix `/run/media/milosvasic/DATA4TB/Projects/boba` — **14 references** across `WorkingDirectory=`, `ExecStart=`, `ExecStop=`, `EnvironmentFile=` and `Documentation=`. That directory does not exist on this host; the real checkout is `/home/milosvasic/Projects/boba`. Nothing checked it. Such a

unit installs clean, enables clean, passes `systemd-analyze verify`, and then dies at **activation** on the next boot with a bare `CHDIR` failure — the §11.4.108 source-looks-fine / runtime-broken gap.

Why templating rather than correcting the constant. A hardcoded path *is* the defect. Replacing one wrong constant with one right constant leaves the same trap armed for the next checkout location — a worktree, a second track under `/mnt/trackN`, a clone on another host. The token removes the class.

The tradeoff, stated plainly. Substitution requires a real file, so a symlink is impossible: systemd would read the raw token and fail.

Cost	Installed units no longer track repo edits live. After editing any unit, or after a `git pull` that changes one, you must re-run `boba-svc install`.
Benefit	Units are checkout-location-independent. The same repo works from any path with no edit.

The cost is one idempotent command; the benefit is that the defect class cannot recur. The drift is not left to vigilance: `install` reports `changed` vs `already current` per unit, and `tests/pre_build/test_systemd_unit_paths.sh` **ARM 2** asserts against the *installed* copies — the bytes systemd actually reads — not merely the repo templates. `install` also **fails closed**: if any token survives substitution, it refuses to install that unit rather than shipping one that dies at activation.

The orphaned timer (found and fixed 2026-09-01)

`boba-resource-pressure-check.timer` was wired to **nothing**. It is deliberately absent from `boba.target`'s `Wants=` (so monitoring survives `boba-svc down`), but `boba-svc enable` only enabled `boba.target`, so the timer's own `WantedBy=timers.target` was never realised — verified by the absence of any symlink in `~/.config/systemd/user/timers.target.wants/`. The hourly forced-logout-precursor probe that task #77 / BOB-076 exists to run would never have fired after a reboot: exactly the monitoring gap that incident argued was self-defeating.

`boba-svc enable` now enables an `ENABLE_UNITS` list — `boba.target` **and** the timer. The timer stays outside `boba.target`, so `down` still leaves monitoring running.

`boba-stack.service` and `boba-webui-bridge.service` correctly remain `disabled`: the target's `Wants=` is what starts them, and enabling them individually would only add a redundant second path to the same thing.

PROVEN vs UNPROVEN — reboot survival (§11.4.6)

This distinction is load-bearing. **Reboot survival has NOT been verified**, because verifying it requires a reboot, and rebooting this host is forbidden (CONST-033).

PROVEN — observed directly on 2026-09-01, each with captured command output:

Claim	Evidence
Units installed	`boba-svc install` → 5 changed; re-run → `0 changed, 5 already current` (idempotent)
Substitution correct in what systemd reads	`systemctl --user cat boba.target` shows `/home/milosvasic/Projects/boba`, no token
Units are syntactically valid	`systemd-analyze --user verify` → `rc=0` for all five
`boba.target` enabled	`systemctl --user is-enabled boba.target` → `enabled`; symlink in `default.target.wants/`
Timer enabled	`is-enabled` → `enabled`; symlink in `timers.target.wants/`
Linger enabled	`loginctl show-user milosvasic --property=Linger` → `Linger=yes`
Every referenced path exists	`test_systemd_unit_paths.sh` → 38 paths, 0 failures, exit 0
The guard genuinely catches the defect	Paired mutation: reintroduced bad path → FAIL; unsubstituted token → FAIL; unmutated → PASS

Claim	Evidence
`ExecStart` referents resolve	`start.sh`, `stop.sh`, `webui-bridge`, challenge script all exist and are executable; `./start.sh` resolves from `WorkingDirectory`; `--no-build` is a real flag (`start.sh:1244`)

UNPROVEN — requires an operator-attended reboot:

1. **That the stack actually comes up after a host reboot.** Every *precondition* is proven; the end-to-end boot path is not. Enabled + valid + linger-on is necessary, not sufficient.
2. **That `boba-stack.service` succeeds when systemd runs it.** The unit is valid and `./start.sh --no-build` resolves, but it was deliberately **not executed** — another work stream was actively editing `start.sh`. Static resolution is not execution.
3. **That the ordering holds under real boot conditions** — `network-online.target` readiness, the container runtime being up when `start.sh` runs, first-boot image pulls inside `TimeoutStartSec=300`.
4. **That the timer fires on its `OnBootSec=5min` schedule.**

To close these, an operator reboots and then runs:

```
systemctl --user status boba.target boba-stack.service boba-webui-
        bridge.service
systemctl --user list-timers 'boba*'
journalctl --user -u boba-stack.service -b
bash scripts/boba-svc.sh health
bash tests/pre_build/test_systemd_unit_paths.sh
```

Until that output exists, the honest claim is: **the units are installed, enabled, valid, and every path they reference resolves — reboot survival itself is untested.**